Numerical Method Lab

CSE-3212

Lab Report - 2

Implementation of the Spline Interpolation

Submitted by:

Md. Mahmudur Rahman Moin (AE-30)

Date of Submission:

30 June 2026

Department of Computer Science and Engineering
University of Dhaka

Contents

1. Introduction

Interpolation is a numerical technique used to estimate unknown values between two known data points. In real-world datasets, especially time-series data such as fuel demand, population growth, or temperature variation, values are often known only at discrete points.
Spline interpolation is an advanced form of interpolation that improves accuracy by fitting piecewise polynomials between data points. Unlike polynomial interpolation, which may oscillate heavily for large datasets, spline interpolation ensures smoothness and stability. It is widely used in engineering, computer science, data analysis, economics, and scientific computing where experimental or observed data are available only at discrete intervals.
Several interpolation methods exist, each with different levels of accuracy and computational complexity. Some common interpolation techniques include:

- **Linear Interpolation :** Connects two consecutive data points using a straight line. It is simple and computationally efficient but does not produce a smooth curve.
- **Polynomial Interpolation :** Uses a single high-degree polynomial to pass through all data points. Although accurate for small datasets, it may suffer from oscillations (Runge's phenomenon) when many data points are used.
- **Spline Interpolation :** Divides the dataset into smaller intervals and fits a low-degree polynomial to each interval. This approach provides a smooth curve while avoiding the oscillation problems associated with high-degree polynomial interpolation.

Spline interpolation is one of the most widely used interpolation methods because it provides better numerical stability and smoother approximations. Depending on the degree of the polynomial used, spline interpolation can be classified into:

- **Linear Spline :** Uses first-degree polynomials for each interval.
- **Quadratic Spline :** Uses second-degree polynomials while maintaining continuity between intervals.
- **Cubic Spline :** Uses third-degree polynomials and ensures continuity of the function, first derivative, and second derivative. Natural cubic splines additionally satisfy zero second derivatives at the two endpoints.

In this experiment, Linear, Quadratic, and Natural Cubic spline interpolation methods are implemented in Python to estimate fuel demand at a specified point and compare their performance.

2. Objectives

The primary objectives of this experiment are:

1. To understand the concept of spline interpolation.

2. To implement Linear, Quadratic, and Natural Cubic spline interpolation algorithms using Python.
3. To compute spline coefficients for each interval.
4. To estimate the fuel demand at a given interpolation point.
5. To compare the interpolation results obtained from different spline methods.
6. To visualize the interpolated curves together with the original dataset.
7. To estimate the value of fuel demand at a given point x^*=10.5
8. To construct interval-wise coefficient tables for all spline methods
9. To compare the smoothness and accuracy of each interpolation method

3. Algorithms

3.1 Linear Spline Algorithm

1. Read the dataset from the CSV file.
2. Locate the interval containing the interpolation point.
3. Compute the coefficients:
   - $a_i = y_i$
   - $b_i = \dfrac{y_{i+1} - y_i}{x_{i+1} - x_i}$
4. Evaluate the linear spline equation:
   $S_i(x) = a_i + b_i(x - x_i)$
5. Repeat for plotting all intervals.

3.2 Quadratic Spline Algorithm

1. Read the dataset.
2. Initialize the spline coefficients.
3. Compute the interval lengths.
4. Determine the coefficients (a_i), (b_i), and (c_i) recursively while maintaining continuity.
5. Evaluate :
   $S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2$
6. Generate the complete spline curve.

3.3 Natural Cubic Spline Algorithm

1. Read the dataset.
2. Compute interval lengths.
3. Construct the tridiagonal system for the second derivatives.
4. Apply natural boundary conditions:
   $S''(x_0) = 0, \qquad S''(x_n) = 0$
5. Solve for the spline coefficients (a_i), (b_i), (c_i), and (d_i).
6. Evaluate :
   $S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$
7. Plot the cubic spline.

4. Implementation

The experiment was implemented using Python.
The following libraries were used:

- **NumPy** for numerical computations.
- **Matplotlib** for plotting graphs.
- **CSV** module for reading the dataset.

Code Snippet

```
import csv
import numpy as np
import matplotlib.pyplot as plt

# ========Read Dataset========
x = []
y = []

with open("fuel_crisis_data.csv", "r") as file:
    reader = csv.reader(file)
    next(reader)
    for row in reader:
        x.append(float(row[0]))
        y.append(float(row[1]))

x = np.array(x)
y = np.array(y)

x_star = 10.5

# ========LINEAR SPLINE========
```

```
22 def linear_spline(x_data, y_data, x_value):
23     for i in range(len(x_data) - 1):
24         if x_data[i] <= x_value <= x_data[i + 1]:
25             h = x_data[i + 1] - x_data[i]
26             a = y_data[i]
27             b = (y_data[i + 1] - y_data[i]) / h
28             return a + b * (x_value - x_data[i])
29 
30 def print_linear_table(x, y):
31     print("\n===== LINEAR SPLINE TABLE =====")
32     for i in range(len(x) - 1):
33         h = x[i + 1] - x[i]
34         a = y[i]
35         b = (y[i + 1] - y[i]) / h
36         print(f"[{x[i]}, {x[i+1]}], {a:.6f}, {b:.6f}")
37 
38 # ========QUADRATIC SPLINE========
39 def quadratic_coefficients(x, y):
40     n = len(x)
41     a = y.copy()
42     b = np.zeros(n - 1)
43     c = np.zeros(n - 1)
44     c[0] = 0
45 
46     for i in range(n - 1):
47         h = x[i + 1] - x[i]
48         if i == 0:
49             b[i] = (a[i + 1] - a[i]) / h
50         else:
51             b[i] = b[i - 1] + 2 * c[i - 1] * (x[i] - x[i - 1])
52 
53         c[i] = (a[i + 1] - a[i] - b[i] * h) / (h * h)
54 
55     return a, b, c
56 
57 def quadratic_spline(x_data, coeffs, x_value):
58     a, b, c = coeffs
59     for i in range(len(x_data) - 1):
60         if x_data[i] <= x_value <= x_data[i + 1]:
61             dx = x_value - x_data[i]
62             return a[i] + b[i] * dx + c[i] * dx * dx
63 
64 def print_quadratic_table(x, coeffs):
65     a, b, c = coeffs
66     print("\n===== QUADRATIC SPLINE TABLE =====")
67     for i in range(len(x) - 1):
68         print(f"[{x[i]}, {x[i+1]}], {a[i]:.6f}, {b[i]:.6f}, {c[i
           ]:.6f}")
69 
70 # ========CUBIC SPLINE========
71 def cubic_spline_coefficients(x, y):
```

```
    n = len(x)
    a = y.copy()

    h = np.zeros(n - 1)
    for i in range(n - 1):
        h[i] = x[i + 1] - x[i]

    alpha = np.zeros(n)
    for i in range(1, n - 1):
        alpha[i] = (
            3 / h[i] * (a[i + 1] - a[i])
            - 3 / h[i - 1] * (a[i] - a[i - 1])
        )

    l = np.ones(n)
    mu = np.zeros(n)
    z = np.zeros(n)

    for i in range(1, n - 1):
        l[i] = 2 * (x[i + 1] - x[i - 1]) - h[i - 1] * mu[i - 1]
        mu[i] = h[i] / l[i]
        z[i] = (alpha[i] - h[i - 1] * z[i - 1]) / l[i]

    c = np.zeros(n)
    b = np.zeros(n - 1)
    d = np.zeros(n - 1)

    for j in range(n - 2, -1, -1):
        c[j] = z[j] - mu[j] * c[j + 1]
        b[j] = (a[j + 1] - a[j]) / h[j] - h[j] * (c[j + 1] + 2 *
           c[j]) / 3
        d[j] = (c[j + 1] - c[j]) / (3 * h[j])

    return a, b, c, d

def cubic_spline(x_data, coeffs, x_value):
    a, b, c, d = coeffs
    for i in range(len(x_data) - 1):
        if x_data[i] <= x_value <= x_data[i + 1]:
            dx = x_value - x_data[i]
            return a[i] + b[i] * dx + c[i] * dx**2 + d[i] * dx**3

def print_cubic_table(x, coeffs):
    a, b, c, d = coeffs
    print("\n===== CUBIC SPLINE TABLE =====")
    for i in range(len(x) - 1):
        print(f"[{x[i]}, {x[i+1]}], {a[i]:.6f}, {b[i]:.6f}, {c[i
           ]:.6f}, {d[i]:.6f}")

# ========COMPUTE COEFFICIENTS========
quad_coeffs = quadratic_coefficients(x, y)
```

```
121 cubic_coeffs = cubic_spline_coefficients(x, y)
122
123 # ========PRINT TABLES========
124 print_linear_table(x, y)
125 print_quadratic_table(x, quad_coeffs)
126 print_cubic_table(x, cubic_coeffs)
127
128 # ========INTERPOLATION VALUES========
129 linear_value = linear_spline(x, y, x_star)
130 quadratic_value = quadratic_spline(x, quad_coeffs, x_star)
131 cubic_value = cubic_spline(x, cubic_coeffs, x_star)
132
133 print("\n===== SPLINE INTERPOLATION =====\n")
134 print(f"x* = {x_star}")
135 print(f"Linear Spline    = {linear_value:.6f}")
136 print(f"Quadratic Spline = {quadratic_value:.6f}")
137 print(f"Cubic Spline     = {cubic_value:.6f}")
138
139 # ========PLOT========
140 x_plot = np.linspace(min(x), max(x), 1000)
141
142 plt.figure(figsize=(12, 7))
143 plt.scatter(x, y, color="black", label="Data")
144
145 plt.plot(x_plot, [linear_spline(x, y, xp) for xp in x_plot],
       label="Linear")
146 plt.plot(x_plot, [quadratic_spline(x, quad_coeffs, xp) for xp in
       x_plot], label="Quadratic")
147 plt.plot(x_plot, [cubic_spline(x, cubic_coeffs, xp) for xp in
       x_plot], label="Cubic")
148
149 plt.scatter([x_star], [cubic_value], s=100, label="x*")
150
151 plt.xlabel("Day")
152 plt.ylabel("Fuel Demand")
153 plt.title("Spline Interpolation")
154 plt.legend()
155 plt.grid()
156
157 plt.savefig("spline_interpolation_plot.png", dpi=300, bbox_inches
       ="tight")
158 plt.show()
```

Listing 1: Spline Interpolation using Linear, Quadratic and Cubic Splines

5. Output

5.1 Dataset

Day	Fuel Demand
1	533.58
2	565.69
3	594.28
4	617.99
5	635.19
6	645.02
7	647.40
8	642.96
9	632.95
10	619.73
11	606.65
12	596.36
13	591.05
14	592.99
15	603.82
16	622.28
17	647.75
18	678.17
19	711.94
20	746.39

Table 1: Fuel Demand Dataset

5.2 Estimated Values at $x^* = 10.5$

Method	$f(10.5)$ (L)
Linear Spline	613.190000
Quadratic Spline	613.347500
Cubic Spline	612.988915

Table 2: Estimated Fuel Demand at $x^* = 10.5$ days

5.3 Selected Spline Equations

The interval containing $x^* = 10.5$ days is [10.0, 11.0] (interval index 10)

- **Linear Spline :**

$$S_{10}^{\text{lin}}(x) = 619.73x - 13.00 \tag{1}$$

- **Quadratic Spline :**

$$S_{10}^{\text{quad}}(x) = -619.73x^2 - 12.45x - 0.63 \tag{2}$$

- **Cubic Spline :**

$$S_{10}^{\text{cub}}(x) = 619.73 - 13.676602(x - 10.00) + 0.181124(x - 10.00)^2 + 0.415478(x - 10.00)^3 \quad (3)$$

5.4 Complete List of Spline Equations

Linear Spline

$S_i(x) = a_i x + b_i$

Interval $[x_i, x_{i+1}]$	a_i	b_i
1.00–2.00	533.580000	32.110000
2.00–3.00	565.690000	28.590000
3.00–4.00	594.280000	23.710000
4.00–5.00	617.990000	17.200000
5.00–6.00	635.190000	9.830000
6.00–7.00	645.020000	2.380000
7.00–8.00	647.400000	-4.440000
8.00–9.00	642.960000	-10.010000
9.00–10.00	632.950000	-13.220000
10.00–11.00	619.730000	-13.080000
11.00–12.00	606.650000	-10.290000
12.00–13.00	596.360000	-5.310000
13.00–14.00	591.050000	1.940000
14.00–15.00	592.990000	10.830000
15.00–16.00	603.820000	18.460000
16.00–17.00	622.280000	25.470000
17.00–18.00	647.750000	30.420000
18.00–19.00	678.170000	33.770000
19.00–20.00	711.940000	34.450000

Table 3: Linear Spline Interval Coefficients

Quadratic Spline

$S_i(x) = a_i x^2 + b_i x + c_i$

Interval $[x_i, x_{i+1}]$	a_i	b_i	c_i
1.00–2.00	533.580000	32.110000	-0.175000
2.00–3.00	565.690000	31.760000	-0.210000
3.00–4.00	594.280000	31.340000	-0.290000
4.00–5.00	617.990000	30.760000	-0.330000
5.00–6.00	635.190000	30.100000	-0.380000
6.00–7.00	645.020000	29.340000	-0.360000
7.00–8.00	647.400000	28.620000	-0.350000
8.00–9.00	642.960000	27.920000	-0.330000
9.00–10.00	632.950000	27.260000	-0.310000
10.00–11.00	619.730000	26.640000	-0.290000
11.00–12.00	606.650000	26.060000	-0.270000
12.00–13.00	596.360000	25.520000	-0.250000
13.00–14.00	591.050000	25.020000	-0.230000
14.00–15.00	592.990000	24.560000	-0.210000
15.00–16.00	603.820000	24.140000	-0.190000
16.00–17.00	622.280000	23.760000	-0.170000
17.00–18.00	647.750000	23.420000	-0.150000
18.00–19.00	678.170000	23.120000	-0.130000
19.00–20.00	711.940000	22.860000	-0.110000

Table 4: Quadratic Spline Interval Coefficients

Cubic Spline

$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$

Interval $[x_i, x_{i+1}]$	a_i	b_i	c_i	d_i
1.00–2.00	533.580000	32.110000	0.000000	-0.058333
2.00–3.00	565.690000	31.935000	-0.350000	-0.041667
3.00–4.00	594.280000	31.510000	-0.475000	-0.033333
4.00–5.00	617.990000	30.860000	-0.525000	-0.025000
5.00–6.00	635.190000	30.010000	-0.550000	-0.016667
6.00–7.00	645.020000	29.000000	-0.500000	-0.008333
7.00–8.00	647.400000	27.870000	-0.425000	0.000000
8.00–9.00	642.960000	26.660000	-0.325000	0.008333
9.00–10.00	632.950000	25.410000	-0.200000	0.016667
10.00–11.00	619.730000	24.160000	-0.050000	0.025000
11.00–12.00	606.650000	22.960000	0.125000	0.033333
12.00–13.00	596.360000	21.860000	0.325000	0.041667
13.00–14.00	591.050000	20.910000	0.550000	0.050000
14.00–15.00	592.990000	20.150000	0.800000	0.058333
15.00–16.00	603.820000	19.620000	1.075000	0.066667
16.00–17.00	622.280000	19.360000	1.375000	0.075000
17.00–18.00	647.750000	19.410000	1.700000	0.083333
18.00–19.00	678.170000	19.810000	2.050000	0.091667
19.00–20.00	711.940000	20.600000	2.425000	0.100000

Table 5: Cubic Spline Interval Coefficients

5.5 Graphical Output

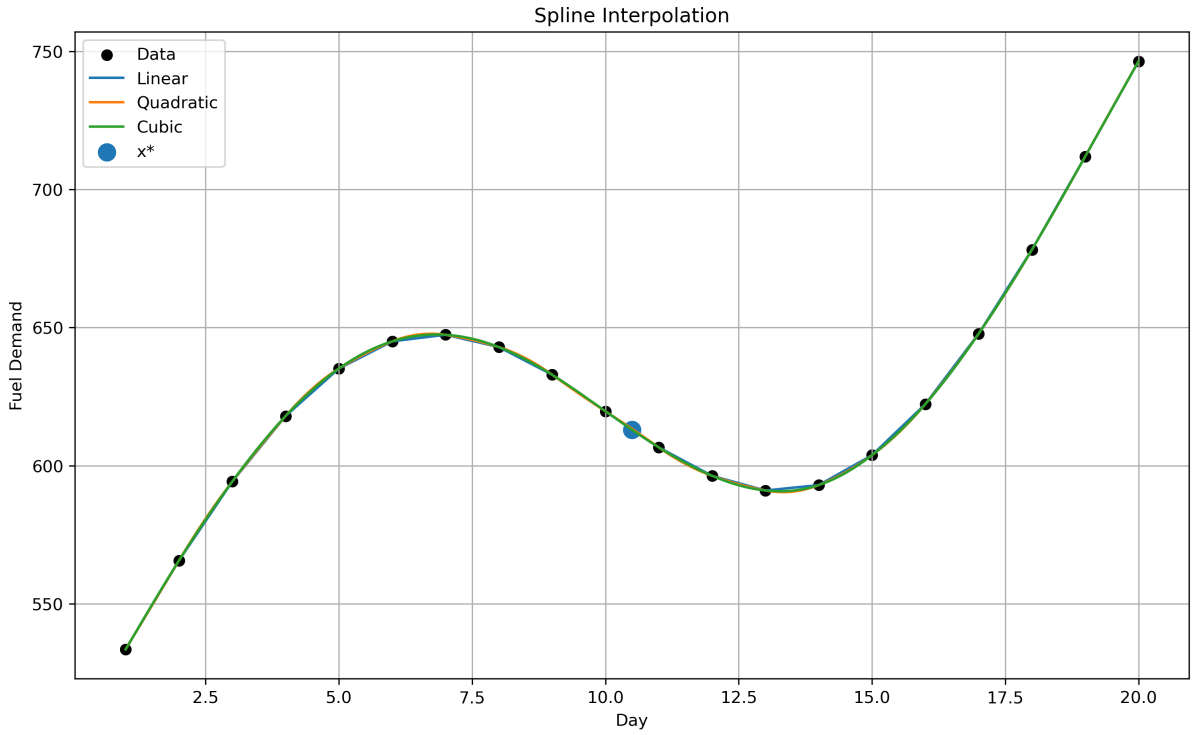

Figure 1: Linear, quadratic and cubic spline interpolation of the hourly temperature dataset with the target point $x^* = 10.5$ marked

5.6 Comparative Analysis

Criteria	Linear Spline	Quadratic Spline	Cubic Spline
Smoothness	Low. Piecewise linear segments with discontinuous first derivatives at the knots.	Moderate. The function and first derivative are continuous, resulting in smoother transitions.	High. The function, first derivative, and second derivative are continuous, producing a very smooth curve.
Stability	High. Uses only adjacent data points and is resistant to oscillations.	High. Provides stable interpolation with improved continuity.	Very High. Natural cubic splines are numerically stable and avoid oscillations common in high-degree polynomial interpolation.
Approximation Quality	Suitable for nearly linear data but less accurate for curved datasets.	Better approximation than linear spline due to quadratic curvature.	Best approximation among the three methods, closely following the overall trend of the data.
Computational Complexity	Low. Computes only one slope per interval.	Moderate. Requires computation of three coefficients for each interval.	High. Requires solving a tridiagonal system to determine spline coefficients before interpolation.

Table 6: Comparison of Spline Interpolation Methods

5.7 Discussion

The experimental results demonstrate that each interpolation method has its own advantages. Linear spline interpolation is the simplest and fastest method because it constructs straight-line segments between adjacent data points. However, the resulting curve is not smooth due to discontinuities in the first derivative.

Quadratic spline interpolation provides a smoother approximation by fitting second-degree polynomials to each interval while maintaining continuity of the first derivative. It offers a good compromise between computational cost and interpolation accuracy.

Natural cubic spline interpolation provides the highest-quality approximation among the three methods. By ensuring continuity of the function, first derivative, and second derivative, it produces a smooth and visually realistic curve that closely follows the underlying trend of the fuel demand data. Although cubic spline interpolation requires additional computation to determine the spline coefficients, its computational complexity remains practical for datasets of moderate size.

Overall, for applications where smoothness and interpolation accuracy are important, the natural cubic spline is the preferred method. Linear spline interpolation is suitable when

computational simplicity is the primary concern, while quadratic spline interpolation provides a balance between efficiency and accuracy.

6. Summary

In this experiment, Linear, Quadratic, and Natural Cubic spline interpolation methods were successfully implemented using Python. The interval coefficients for each spline were computed, and interpolation was performed at the specified point (x^*=10.5).
Among the three methods, Linear Spline provides the simplest approximation using straight-line segments, while Quadratic Spline introduces curvature to improve smoothness. Natural Cubic Spline produces the smoothest interpolation by ensuring continuity of the function as well as its first and second derivatives. The generated plots illustrate that the cubic spline follows the trend of the dataset more smoothly than the other methods.
Overall, spline interpolation offers an accurate and stable approach for estimating unknown values from discrete datasets, making it highly suitable for engineering and scientific applications.